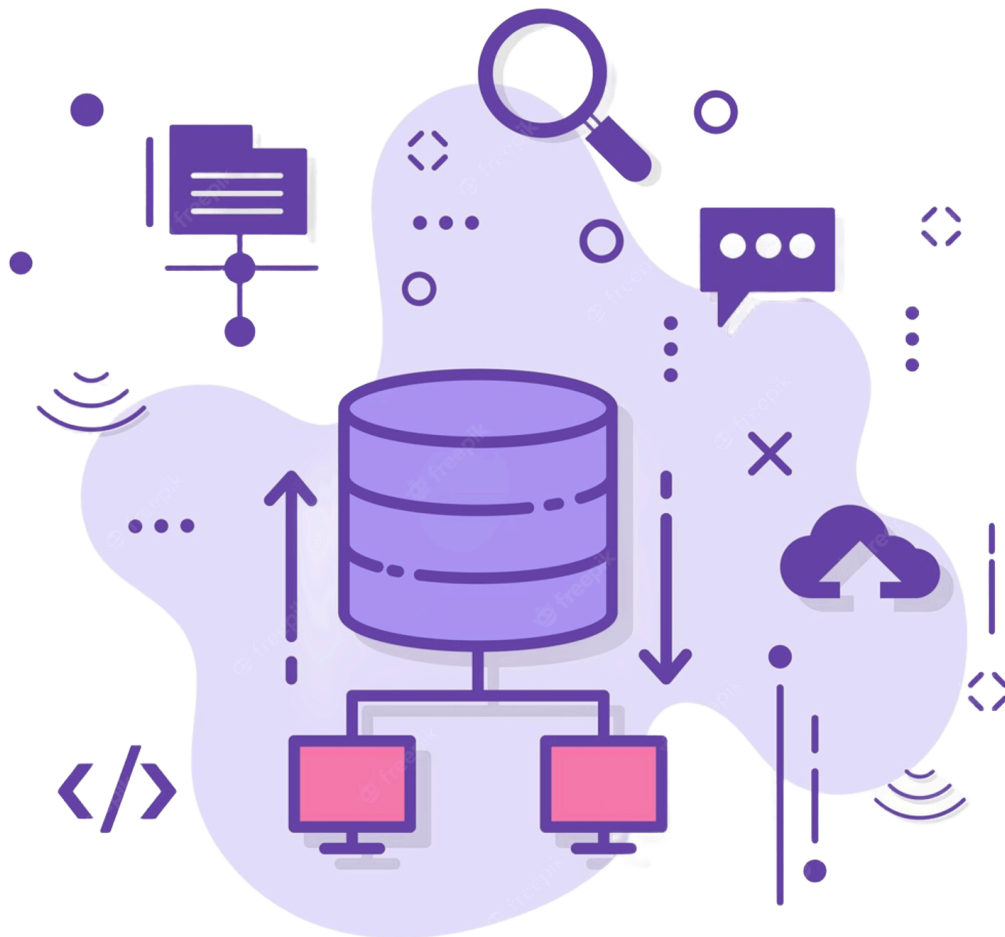


TRABAJO PRÁCTICO INTEGRADOR

FINAL BASE DE DATOS



Docentes:

- Bustos Aguiar Soledad
- Reynoso Chocobar Juan Matías
- Manuel Baquinzay

Alumnos:

- | | |
|-----------------------------|-----------|
| - Juan Agustin Gallo | MU: 01731 |
| - Bravo Nicolas | MU: 01805 |
| - Quintero Facundo Joel | MU: 01847 |
| - Moreira Joaquín | MU: 01884 |
| - Escalante Judith Griselda | MU: 01862 |
| - Yapura Fuenzalida Víctor | MU: 01913 |

ETAPA I

Documentación de Decisiones de Diseño Técnico

1. Arquitectura General del Sistema

El presente diseño físico ha sido estructurado para resolver la problemática de un sistema de gestión de stock de ropa deportiva. Para garantizar la escalabilidad, el rendimiento y la integridad de los datos, el modelo relacional se dividió lógicamente en tres grandes bloques: Entidades Maestras (Catálogos), Núcleo de Productos/Variantes, y Entidades Transaccionales. Cada tabla fue normalizada hasta la Tercera Forma Normal (3FN) para evitar redundancias y anomalías de actualización.

2. Justificación del Bloque de Entidades Maestras (Catálogos)

Estas entidades almacenan la información estática que da contexto al negocio. Su separación es fundamental para cumplir con la normalización:

- **Marcas y Categorías:** En lugar de escribir el nombre de la marca (ej. "Nike") o la categoría ("Calzado") directamente en cada producto, se crearon tablas independientes. Esto evita la redundancia de datos (cumpliendo la 2FN y 3FN) y previene errores de tipeo. Si una marca cambia de nombre, se actualiza en un solo registro y se refleja en todo el sistema.
- **Proveedores:** Se aisló la información de los proveedores (Razón social, CUIT, contacto) en su propia tabla con el CUIT como restricción **UNIQUE**. Esto asegura que los datos fiscales no se repitan y permite que un mismo proveedor abastezca múltiples productos sin duplicar su información de contacto.
- **Sucursales:** Dado que el sistema simulará la gestión de grandes volúmenes de datos, es imperativo manejar múltiples puntos de venta o depósitos. Esta tabla centraliza las ubicaciones físicas, permitiendo escalar el negocio sin alterar la lógica transaccional.
- **Empleados:** Se separó la información del personal en su propia tabla. Esto es crucial no solo por normalización, sino para preparar la base de datos para la Etapa IV (Seguridad y Roles). Al tener un **id_empleado**, podremos auditar con precisión qué usuario realizó cada movimiento de inventario sin repetir sus datos personales en cada transacción.

3. Justificación del Bloque de Productos y Variantes

El modelado de la indumentaria deportiva presenta el desafío de las variantes físicas. Un producto conceptual tiene múltiples instancias reales.

- **Productos:** Esta tabla actúa como la cabecera conceptual. Almacena únicamente los atributos que son comunes a todas las variantes de ese artículo (Nombre, Descripción, Precio base, y las claves foráneas hacia **Marcas** y **Categorías**).
- **Talles y Colores:** Se diseñaron como catálogos independientes. Si hubiéramos incluido las columnas "talle" y "color" en la tabla **Productos**, habríamos generado una dependencia funcional transitiva y repetido el nombre y precio del producto por cada talle existente, violando la 3FN.
- **Producto_Variante:** Esta es la entidad asociativa clave del sistema. Representa el artículo físico real (el SKU o Código de Barras) que se encuentra en la estantería. Conecta un **id_producto** con un **id_talle** y un **id_color** específicos. Esta decisión de diseño garantiza que el inventario se gestione con total precisión granular, sin redundancias.

4. Justificación del Bloque Transaccional

Este bloque fue diseñado para soportar el crecimiento significativo de datos y el análisis de rendimiento requerido en etapas posteriores.

- **Inventario:** Actúa como una "foto" del stock actual. Su clave primaria es compuesta (**id_sucursal**, **id_variante**), lo que asegura una dependencia funcional completa (2FN). La cantidad disponible depende de la combinación exacta de en qué sucursal está y qué variante específica es, permitiendo consultas ultrarrápidas de disponibilidad sin tener que sumar todo el historial de movimientos.
- **Movimientos (Cabecera):** Registra el evento general (Cuándo ocurrió, qué empleado lo hizo, si fue una Entrada, Salida o Traslado entre sucursales). Separar la cabecera del detalle es vital para cumplir con la Primera Forma Normal (1FN), evitando tener atributos multivaluados (múltiples productos en una sola fila).
- **Detalle_Movimientos:** Registra qué variantes exactas y en qué cantidades se movieron durante una transacción específica. Un punto crítico de diseño aquí fue incluir el atributo **precio_unitario**. Esto es necesario porque el precio de un producto cambia con el tiempo; al guardarlo en el detalle del movimiento, "congelamos" el valor al momento exacto de la operación, garantizando la integridad histórica de la información contable y evitando dependencias transitivas hacia el precio actual del catálogo.

5. Integridad de Datos y Restricciones de Dominio

Para asegurar la calidad y consistencia de la información directamente en el motor de la base de datos (PostgreSQL), se implementaron las siguientes restricciones de dominio:

- **Restricciones de Unicidad (UNIQUE):** Se aplicaron en campos críticos del negocio para impedir la duplicación lógica de registros. Esto incluye el **cuit** en la tabla Proveedores, el **documento** en Empleados, las nomenclaturas de Talles y Colores, y el **codigo_barras** en la entidad Producto_Variante.
- **Restricciones de Comprobación (CHECK):** Se establecieron reglas de validación estrictas previas a la inserción de datos.
 - **Valores numéricos lógicos:** Los atributos monetarios (**precio_venta_actual**, **precio_unitario**) y las cantidades (**cantidad_disponible**, **cantidad** en movimientos) deben ser estrictamente mayores o iguales a cero. El sistema rechazará cualquier intento de registrar un stock negativo.
 - **Listas de valores permitidos:** El atributo **tipo_movimiento** solo acepta los valores ('Entrada', 'Salida', 'Traslado'). Del mismo modo, el atributo **perfil_acceso** en la tabla Empleados está restringido a ('Administrador', 'Operativo', 'Consulta'). Esto blindo al sistema contra errores de tipeo y estandariza los datos para la futura generación de reportes.

6. Integridad Referencial y Comportamiento ante Eliminaciones

El manejo de las claves foráneas (Foreign Keys) fue diseñado para proteger el historial de operaciones y evitar la existencia de registros huérfanos, definiendo acciones específicas mediante **ON DELETE** y **ON UPDATE**:

- **Preservación Histórica (ON DELETE RESTRICT):** Es la regla predominante en el modelo, aplicada para proteger la auditoría transaccional. Por ejemplo, el sistema denegará la eliminación de una **Sucursal**, de un **Empleado** o de un **Producto** si existen movimientos de stock o inventarios asociados a ellos. Esto garantiza que el historial contable y logístico nunca se corrompa por borrados accidentales de catálogos.
- **Eliminación en Cascada (ON DELETE CASCADE):** Se implementó exclusivamente en relaciones de composición o dependencia fuerte. Por ejemplo, si se elimina un registro en **Movimientos** (la cabecera), sus **Detalle_Movimientos** asociados se eliminarán automáticamente. Lo mismo ocurre entre **Producto_Variante** y el **Inventario**: si una variante se da de baja definitiva del sistema, sus registros de stock actual desaparecerán para no dejar datos huérfanos inservibles.



- **Actualización Propagada (ON UPDATE CASCADE):** Se aplicó como regla universal en todas las claves foráneas. Esto asegura que si, por una operación de mantenimiento de base de datos, el identificador (ID) de un registro maestro llegara a modificarse, dicho cambio se propagará automáticamente hacia todas las tablas que dependan de él, manteniendo intactos los vínculos relacionales sin intervención manual.

7. Elección de claves primarias En la mayoría de las entidades se optó por utilizar claves primarias subrogadas (identificadores numéricos autoincrementales) para optimizar el rendimiento del sistema y garantizar la estabilidad, aplicando restricciones de unicidad sobre los identificadores naturales del dominio.

- **PRODUCTOS, CATEGORIAS, MARCAS, TALLES, COLORES y MOVIMIENTOS:** se utilizaron claves primarias subrogadas (`id_producto`, `id_categoria`, etc.) ya que los nombres o descripciones pueden estar sujetos a cambios o modificaciones, garantizando así un identificador estable e inmutable a nivel de sistema.
- **PROVEEDORES:** se utilizó `id_proveedor` como clave primaria por eficiencia, pero se aplicó una restricción **UNIQUE** al atributo `cuit` por ser el identificador fiscal natural, único y obligatorio para cada proveedor.
- **EMPLEADOS:** se eligió `id_empleado` como clave primaria, manteniendo `documento` con restricción **UNIQUE** para asegurar que no se dupliquen registros del personal.
- **PRODUCTO_VARIANTE:** se utilizó `id_variante` como clave primaria, con el `codigo_barras` registrado como clave única, ya que cada artículo físico con un talle y color específico debe tener una representación exacta.
- **INVENTARIO:** se utilizó una clave primaria compuesta por `id_sucursal` e `id_variante`. La cantidad disponible de stock no depende de la sucursal sola ni del producto solo, sino de la combinación única de ambos.

En todos los casos se priorizaron atributos que garantizan unicidad, estabilidad y disponibilidad en el sistema.

8. Resolución de relaciones N:M Para la relación ALMACENA entre **SUCURSALES** y **PRODUCTO_VARIANTE** se creó la tabla **INVENTARIO**, cuya clave primaria es compuesta (`id_sucursal`, `id_variante`). Esta combinación permite identificar de manera única la disponibilidad de cada variante física en cada ubicación. Ambos atributos actúan además como claves foráneas hacia sus respectivas tablas.

Para la relación CONTIENE entre **MOVIMIENTOS** y **PRODUCTO_VARIANTE** se creó la tabla **DETALLE_MOVIMIENTOS**. Aunque conceptualmente resuelve la

cardinalidad N:M, se optó por agregarle una clave primaria propia (**id_detalle**) para facilitar su manejo transaccional en el historial, manteniendo **id_movimiento** e **id_variante** como claves foráneas.

9. Acciones referenciales

- **Inventario** → **ON DELETE RESTRICT**: no permitís borrar una sucursal si tiene stock registrado (evita dejar inventario "huérfano" en el sistema).
 - **ON UPDATE CASCADE**: si cambia el id de sucursal, se propaga a INVENTARIO.
- **Detalle_Movimientos** → **ON DELETE CASCADE**: si se elimina o anula un movimiento base (la cabecera), se eliminan automáticamente todos sus detalles (no pueden existir renglones de detalle sin su movimiento de origen).
 - **ON UPDATE CASCADE**: si cambia el id del movimiento, se propaga a DETALLE_MOVIMIENTOS.
- **Producto_Variante** → **ON DELETE RESTRICT**: no se puede eliminar un producto base, un talle o un color si ya están siendo utilizados para definir una variante física existente.
- **Movimientos** → **ON DELETE RESTRICT**: una sucursal o un empleado no pueden eliminarse del sistema si existen transacciones históricas registradas bajo su nombre.

10. Atributos multivaluados Los atributos multivaluados del DER fueron transformados en tablas auxiliares para cumplir con la primera forma normal. En **PRODUCTOS**, las variaciones físicas del artículo (talle y color) se modelaron como tablas independientes: **TALLES** y **COLORES**. Para evitar la redundancia, la asociación se resolvió a través de la tabla **PRODUCTO_VARIANTE**, en donde la clave primaria de las tablas auxiliares (**id_talle**, **id_color**) actúan como claves foráneas para definir la versión final y única de la indumentaria deportiva.

- **Transformacion DER a Modelo relacional**

ENTIDADES	
Marcas	Fuerte
Categorias	Fuerte
Proveedores	Fuerte
Sucursales	Fuerte
Empleados	Fuerte
Productos	Fuerte
Colores	Fuerte



Talles	Fuerte
Producto_variante	Debil
Movimientos	Fuerte
Detalle_movimientos	Debil
RELACIONES	
Almacena	N:M
Contiene	N:M
Agrupar	1:N
Pertenece	1:N
Posee	1:N
Aplica	1:N
Distingue	1:N
Abastece	1:N
Destino	1:N
Registra	1:N
ATRIBUTOS ESPECIALES	
variaciones físicas (talle, color)	Multivaluado → Descompuesto en entidades auxiliares (TALLE, COLOR) y asociativa (PRODUCTO_VARIANTE)
nombre_empleado	Compuesto → Descompuesto en nombre y apellido

Entidades

MARCA	
#id_marca	nombre

CATEGORIA		
#id_categoria	nombre	descripcion

SUCURSAL			
#id_sucursal	nombre	direccion	Ciudad

PROVEEDOR



#id_proveedor	razon_social	email	cuit	telefono
---------------	--------------	-------	------	----------

EMPLEADO

#id_empleado	documento	nombre	apellid o	perfil_acceso
--------------	-----------	--------	--------------	---------------

PRODUCTO

#id_product o	nombre	descripcio n	Precio_venta_actu al	*id_marc a	*id_categori a
------------------	--------	-----------------	-------------------------	---------------	-------------------

COLOR

#id_color	Nombre_color
-----------	--------------

TALLE

#id_talle	nomenclatura
-----------	--------------

PRODUCTO_VARIANTE

#id_variant e	Código_barra s	*id_product o	*id_talle	*id_colo r
------------------	-------------------	------------------	-----------	---------------

PRODUCTO

#id_movimi ento	Fecha _hora	Tipo_movi miento	obseva ciones	*id_pro veedor	*id_sucurs al	*id_emplea do
--------------------	----------------	---------------------	------------------	-------------------	------------------	------------------

DETALLE_MOVIMIENTO

#id_detalle	cantidad	Precio_unitari o	*id_movimient o	*id_variant e
-------------	----------	---------------------	--------------------	------------------

Relaciones

Relación N:M — CONTIENE

Movimiento {N}  CONTIENE  {M} Producto_Variante

CONTIENE(#id_detalle , #*id_movimiento , #*id_variante)

#id_detalle	#*id_movimiento	#*id_variante
-------------	-----------------	---------------

- Relación N:M – ALMACENA

Sucursal {N} ◇ ALMACENA ◇ {N} Producto_Variante

ALMACENA(#*id_sucursal , #*id_variante)	
#*id_sucursal	#*id_variante

- Relación 1:N – AGRUPA

Categoria {1} ◇ AGRUPA ◇ {N} Producto

→ FK: id_categoria en PRODUCTO → CATEGORIA

- Relación 1:N – PERTENECE

Marca {1} ◇ PERTENECE ◇ {N} Producto

→ FK: id_marca en PRODUCTO → MARCA

- Relación 1:N – POSEE

Producto {1} ◇ POSEE ◇ {N} Producto_Variante

→ FK: id_producto en PRODUCTO_VARIANTE → PRODUCTO

- Relación 1:N – APLICA

Talle {1} ◇ APLICA ◇ {N} Producto_Variante

→ FK: id_talle en PRODUCTO_VARIANTE → TALLE

- Relación 1:N – DISTINGUE

Color {1} ◇ DISTINGUE ◇ {N} Producto_Variante

→ FK: id_color en PRODUCTO_VARIANTE → COLOR

- Relación 1:N – ABASTECE



Proveedor {1} ◇ ABASTECE ◇ {N} Movimiento

→ FK: id_proveedor en MOVIMIENTO → PROVEEDOR

- Relación 1:N – DESTINO

Sucursal {1} ◇ DESTINO ◇ {N} Movimiento

→ FK: id_sucursal en MOVIMIENTO → SUCURSAL

- Relación 1:N – REGISTRA

Empleado {1} ◇ REGISTRA ◇ {N} Movimiento

→ FK: id_empleado en MOVIMIENTO → EMPLEADO

● Identificación de tipos de clave

Tabla	PK (tipo)	FK → tabla ref.	Candidata
MARCA	id_marca — simple	—	nombre
CATEGORIA	id_categoria — simple	—	nombre
PROVEEDOR	id_proveedor — simple	—	cuit
SUCURSAL	id_sucursal — simple	—	—
EMPLEADO	id_empleado — simple	—	documento
PRODUCTO	id_producto — simple	id_marca → MARCAid_categoria → CATEGORIA	—

COLOR	id_color simple	—	—	nombre_col or
TALLE	id_talle simple	—	—	nomenclatu ra
PRODUCTO_VARIA NTE	id_variante simple	—	id_producto → PRODUCTOid_talle → TALLEid_color → COLOR	codigo_barr as
MOVIMIENTO	id_movimiento — simple	—	id_proveedor → PROVEEDORid_sucursal → SUCURSALid_empleado → EMPLEADO	—
DETALLE_MOVIMIE NTO	id_detalle simple	—	id_movimiento → MOVIMIENTOid_variante → PRODUCTO_VARIANTE	—

Tabla de Intersección (Relaciones N:M)

ALMACENA	(id_sucursal, id_variante) - compuesta	id_sucursal → SUCURSAL - id_variante → PRODUCTO_VARIANTE	—
CONTIENE	(id_movimiento , id_variante) - compuesta	id_movimiento → MOVIMIENTO - id_variante → PRODUCTO_VARIANTE	—

• Verificación de restricciones de integridad

Tabla	UNIQUE (además PK)	FK → ref. + acciones	NOT NULL	Validación dominio
-------	--------------------------	-------------------------	----------	-----------------------



MARCA	—	—	nombre	nombre no vacío
CATEGORIA	—	—	nombre	nombre no vacío
PROVEEDOR	cuit	—	razon_social, cuit	cuit = 11 dígitos; razon_social no vacía
SUCURSAL	—	—	nombre	nombre/dirección no vacíos
EMPLEADO	documento	—	documento, nombre, apellido	perfil_acceso - IN ('Administrador', 'Operativo', 'Consulta'); documento/nombre no vacíos
PRODUCTO	—	id_marca → MARCAON DELETE RESTRICT, ON UPDATE CASCADEid_categoria → CATEGORIAON DELETE RESTRICT, ON UPDATE CASCADE	nombre, precio_venta_actual, id_marca, id_categoria	precio_venta_actual >= 0; nombre no vacío
COLOR	nombre_color	—	nombre_color	nombre_color no vacío
TALLE	nomenclatura	—	nomenclatura	nomenclatura no vacía



PRODUCTO_VARIANTE	codigo_barras	id_producto → PRODUCTOON DELETE RESTRICT, ON UPDATE CASCADEid_talle → TALLEON DELETE RESTRICT, ON UPDATE CASCADEid_color → COLORON DELETE RESTRICT, ON UPDATE CASCADE	codigo_barras, id_producto, id_talle, id_color	codigo_barras no vacío
MOVIMIENTO	—	id_proveedor → PROVEEDORON DELETE RESTRICT, ON UPDATE CASCADEid_sucursal → SUCURSALON DELETE RESTRICT, ON UPDATE CASCADEid_empleado → EMPLEADOON DELETE RESTRICT, ON UPDATE CASCADE	fecha_hora, tipo_movimiento, id_empleado, id_proveedor, id_sucursal	tipo_movimiento - IN ('Entrada', 'Salida', 'Traslado')
Relaciones (N:M)				

ALMACENA	—	id_sucursal →SUCURSAL ON DELETE RESTRICT, ON UPDATE CASCADE id_variante →PRODUCTO_VA RIANTE ON DELETE CASCADE, ON UPDATE CASCADE	id_sucursal, id_variante, cantidad_disp onible	cantidad_disp onible >= 0
CONTIENE	—	id_movimiento →MOVIMIENTO ON DELETE CASCADE, ON UPDATE CASCADE id_variante →PRODUCTO_VA RIANTE ON DELETE RESTRICT, ON UPDATE CASCADE	id_movimiento , id_variante, cantidad, precio_unitari o	cantidad >= 0; precio_unitari o >= 0

ETAPA II

1- Carga masiva de datos mediante mecanismos adecuados.

Para realizar la carga masiva se utilizó un **script SQL único**, ejecutado desde el **Query Tool de pgAdmin 4**.

El archivo incluyó instrucciones INSERT INTO agrupadas por tabla y ejecutadas dentro de una transacción.



También se hizo uso de TRUNCATE TABLE junto con RESTART IDENTITY CASCADE para de que en caso de las tablas tener datos poder reiniciar las secuencias, evitar duplicados o conflictos con PKs o FKs.

La carga fue realizada siguiendo el orden de dependencia entre las tablas permitiendo que los registros relacionados ya existieran antes de insertar las filas que contenían FKs.

2- Identificación de tablas con crecimiento significativo.

Las tablas que presentan mayor crecimiento son:

Movimientos

Registra cada operación realizada en el sistema:

- Entrada de mercadería.
- Salida por venta.
- Traslado entre sucursales.

Su crecimiento es continuo, porque cada nueva operación genera un nuevo registro.

Actualmente posee un total de 1.000 registros

Detalle_Movimientos

Es la tabla con mayor cantidad con 2.470 registros

Su crecimiento es superior al de **Movimientos**, porque cada movimiento puede contener varias variantes de productos.

Por ejemplo, un movimiento puede registrar simultáneamente:

- Una zapatilla talle 40 color negro.
- Una zapatilla talle 41 color blanco.
- Una remera talle M color azul.

En ese caso, se genera un solo registro en **Movimientos**, pero tres registros en **Detalle_Movimientos**.

Por ello, esta tabla será la de mayor crecimiento a largo plazo.

Inventario

Actualmente posee 96 registros y su crecimiento es controlado.



Su crecimiento depende de la aparición de:

- Nuevas sucursales.
- Nuevos productos.
- Nuevos talles.
- Nuevos colores.
- Nuevas variantes.

Su cantidad puede estimarse como: Cantidad de sucursales $\times$ cantidad de variantes

En nuestro caso particular presenta un total de 96 registros.

Producto_Variante

Puede crecer de forma significativa cuando se incorporan nuevos productos o combinaciones de talle y color.

Sin embargo, su crecimiento suele ser menor que el de las tablas transaccionales, porque una variante se registra una sola vez y luego se reutiliza en muchos movimientos.

3- Cantidad de registros utilizados.

Luego de la ejecución de la carga se obtuvo

	tabla text	cantidad bigint
1	productos	4
2	variantes	48
3	movimient...	1000
4	detalles	2470
5	inventario	96

Donde la tabla productos contiene 4 elementos diferentes donde cada uno posee

- 3 talles válidos por producto
- 4 colores disponibles
- 12 variantes por producto

Lo cual nos termina generando un total de 48 variantes.

La tabla inventario registra una fila por cada combinación entre sucursal y variante, al existir 2 sucursales tenemos que esta posee 96 registros.

Además se generaron 1000 movimientos para representar un gran volumen de operaciones de entrada, salida y traslado.



Cada movimiento contiene uno o más productos por lo que se crearon 2470 registros en `detalles_movimiento`.

4- Estrategia aplicada para la carga.

La estrategia utilizada fue una carga secuencial y controlada mediante un único script SQL. Primero se realizó la inserción de las tablas independientes y luego la de las tablas dependientes. Así pudiendo respetar las claves foráneas y el orden lógico de las relaciones entre las entidades.

Por ejemplo, primero se cargaron las tablas de marcas, categorías, talles y colores. Después se insertaron las sucursales, proveedores y empleados.

A continuación, se cargaron los productos y luego se generaron sus variantes.

Posteriormente, se insertaron los movimientos junto con sus respectivos detalles y, finalmente, se cargó el inventario resultante.

La carga se ejecutó dentro de una sola transacción mediante las sentencias: **BEGIN** y **COMMIT**.

Esto permitió que, en caso de producirse un error durante la carga, PostgreSQL pudiera cancelar toda la operación y evitar que la base de datos quedara parcialmente cargada o con información incompleta.

También se agruparon múltiples registros dentro de una misma instrucción **INSERT**.

Este método resulta más eficiente que ejecutar una instrucción independiente por cada fila, ya que reduce la cantidad de operaciones enviadas al motor de base de datos y disminuye el tiempo total de procesamiento.

Otra alternativa evaluada fue realizar la carga tabla por tabla mediante archivos CSV o TXT separados por tabulaciones. En ese caso, debía utilizarse la herramienta Import/Export Data de pgAdmin 4 y seleccionar manualmente un archivo para cada tabla.

La carga con CSV o TXT también debía respetar el orden de dependencia.

Por ejemplo, no era posible importar primero los productos si todavía no estaban cargadas las marcas y categorías, porque las claves foráneas producirían un error. Del mismo modo, no podían cargarse los detalles de movimientos antes de que existieran los movimientos y las variantes de productos.



Aunque la carga mediante CSV o TXT es válida, presenta algunas desventajas para nuestro proyecto ya que:

- Se debe importar cada tabla de manera individual.
- Es necesario configurar correctamente el delimitador, encabezado, codificación y tratamiento de valores nulos.
- Existe mayor posibilidad de cometer errores al seleccionar columnas o archivos.
- Si una importación falla, las tablas cargadas anteriormente permanecen con datos.
- No se garantiza automáticamente que toda la carga se ejecute como una única operación.
- Es necesario controlar manualmente el orden de las doce tablas.

En cambio, con el script SQL se ejecuta toda la carga desde el Query Tool de pgAdmin 4 en una sola operación.

Por lo tanto, se eligió la carga mediante un único script SQL porque ofrecía mayor control, consistencia y facilidad de ejecución. Además, permitió garantizar que los datos respetarán las restricciones establecidas, como los talles válidos según el tipo de producto, la existencia de proveedores en los movimientos de entrada, las sucursales de origen y destino en los traslados y la disponibilidad de stock.

En conclusión, la carga mediante CSV o TXT hubiera sido adecuada para cargar una tabla aislada o un conjunto pequeño de datos. Sin embargo, para una base relacional compuesta por varias tablas vinculadas, el uso de un único script SQL resultó más seguro, eficiente y fácil de reproducir.

5- Impacto del volumen de datos sobre el rendimiento del sistema.

Con el volumen actual, el sistema no presenta problemas importantes de rendimiento.

Las cantidades cargadas todavía son bajas:

- 1.000 movimientos.
- 2.470 detalles.
- 96 registros de inventario.

PostgreSQL puede procesar este volumen con rapidez.

Aunque a medida que aumenten las operaciones, las tablas **Movimientos** y **Detalle_Movimientos** podrían afectar el rendimiento de consultas como:



- Buscar movimientos por fecha.
- Consultar operaciones de una sucursal.
- Buscar movimientos realizados por un empleado.
- Obtener los productos incluidos en cada movimiento.
- Calcular entradas y salidas por producto.
- Generar reportes históricos.

Las claves primarias y las restricciones **UNIQUE** generan índices automáticamente. Las claves foráneas no siempre crean índices sobre las columnas que las contienen.

Para mejorar el rendimiento futuro sería conveniente crear índices ya que estos índices permiten localizar registros con mayor rapidez, aunque también generan un pequeño costo adicional durante las inserciones y actualizaciones.

En la carga actual, ese costo sería mínimo. Con cientos de miles o millones de registros, sería necesario analizar consultas, índices, planes de ejecución y posiblemente aplicar estrategias adicionales, como particionar los movimientos por fecha.

ETAPA III

Justificación Lógica de las Consultas Avanzadas

Explicación de cómo y por qué se construyó cada una de las soluciones del script 04_Consultas_Reportes.sql:

- **Consulta 1 (Ranking de Recaudación por Marca):** Se armó vinculando las tablas Movimientos, Detalle_Movimientos, Producto_Variante, Productos y Marcas mediante cláusulas INNER JOIN. El porqué fue para identificar qué marcas generaron más ingresos en el último trimestre. Para lograrlo, multiplicamos la cantidad por el precio unitario guardado en el detalle transaccional (evitando usar el precio actual del catálogo por temas de inflación) y agrupamos los valores finales por marca con GROUP BY.
- **Consulta 2 (Identificación de Stock Inactivo):** Se estructuró uniendo el catálogo de variantes con productos, marcas, talles y colores. El porqué fue aislar de forma exacta la indumentaria que no registró rotación comercial en el año. Como un JOIN común solo muestra lo que sí se vendió, decidimos implementar una subconsulta con la regla NOT IN para listar las variantes vendidas y restarlas del catálogo general.
- **Consulta 3 (Rendimiento de Empleados en Sucursal NOA):** Se construyó conectando las cabeceras de movimientos con empleados, sucursales y



detalles de artículos. El porqué fue medir la productividad de los cajeros en Catamarca durante el año 2025. Se ejecutó aplicando un COUNT(DISTINCT id_movimiento) para contabilizar tickets únicos reales (e impedir que se sumen ventas fantasma si un cliente compró varias prendas juntas) junto a un SUM para calcular la recaudación total.

- **Consulta 4 (Alerta Temprana de Stock):** Se armó realizando una unión en cadena de 6 tablas: Inventario, Sucursales, Producto_Variante, Productos, Talles y Colores. El porqué fue generar un reporte operativo de urgencia logística. Se resolvió aplicando un filtro numérico directo con la cláusula WHERE inv.cantidad_disponible < 35 para obtener los artículos críticos ordenados por sucursal.
- **Consulta 5 (Comparativa de Comportamiento por Sucursal):** Se estructuró enlazando el historial de movimientos con los almacenes, detalles, variantes, productos maestros y marcas. El porqué fue analizar y comparar las preferencias de marcas de los clientes según su ubicación geográfica. Se logró aplicando un agrupamiento múltiple en el GROUP BY por el nombre de la sucursal y la marca en simultáneo.

Justificación de la Estrategia de Indexación Físicas

Por qué decidimos optimizar: Al analizar las consultas con la herramienta EXPLAIN ANALYZE, detectamos que el motor de la base de datos realizaba escaneos secuenciales (Seq Scan) recorriendo miles de filas registro por registro. Esto ocurría porque PostgreSQL no indexa automáticamente las claves foráneas, provocando lentitud en los JOINS y en los filtros condicionales a medida que el volumen operativo crecía en producción.

Para solucionar este problema, creamos los siguientes índices estratégicos en el script 05_Indices.sql:

- Creamos el índice **idx_detalle_movimientos_id_mov** para mejorar la búsqueda y el acoplamiento físico entre las cabeceras de movimientos y sus renglones de detalles.
- Creamos el índice **idx_detalle_movimientos_id_var** para mejorar la búsqueda de las variantes de productos específicas dentro del histórico de ventas.
- Creamos el índice **idx_movimientos_id_empleado** para mejorar la búsqueda del legajo de los empleados en los reportes de rendimiento y auditoría.
- Creamos el índice **idx_movimientos_id_sucursal_origen** para mejorar la búsqueda y segmentación geográfica de operaciones por sucursal de origen.
- Creamos el índice **idx_inventario_cantidad_disponible** para mejorar la búsqueda de alertas de bajo stock, permitiendo saltar directo a los registros con menos de 35 unidades.

- Creamos el índice **idx_producto_variante_id_producto** para mejorar la búsqueda de productos maestros asociados a sus variantes intermedias.
- Creamos el índice **idx_producto_variante_id_talle** para mejorar la búsqueda y el filtrado por nomenclaturas de talles en el catálogo.
- Creamos el índice **idx_producto_variante_id_color** para mejorar la búsqueda y cruzamiento por nombres de colores de indumentaria.
- Creamos el índice **idx_productos_id_marca** para mejorar la búsqueda y el agrupamiento de artículos pertenecientes a una misma marca comercial.
- Creamos el índice **idx_productos_id_categoria** para mejorar la búsqueda y clasificación jerárquica por categorías de ropa deportiva.
- Creamos el índice compuesto **idx_movimientos_tipo_fecha** para mejorar la búsqueda en consultas multi-filtro, resolviendo el tipo de movimiento ('Salida') y el rango de fechas en un único paso físico de lectura.

ETAPA IV

Documentación del Modelo de Seguridad (RBAC y Vistas)

Para garantizar la integridad y confidencialidad del ecosistema de base de datos de indumentaria deportiva, se implementó un modelo de Control de Acceso Basado en Roles (RBAC) aplicando el principio de mínimo privilegio.

1. Perfiles de Acceso Creados:

- **Rol Administrador:** Posee delegación total (DML y DDL) sobre el esquema público. Diseñado para tareas de mantenimiento estructural y soporte técnico.
- **Rol Operativo:** Perfil transaccional. Tiene permisos de lectura y escritura (INSERT, UPDATE) únicamente sobre las tablas de flujo de stock (Movimientos, Detalle_Movimientos, Inventario). Su acceso a tablas maestras (Empleados, Proveedores) está restringido a solo lectura (SELECT) para prevenir alteraciones maliciosas en la nómina o cuentas corrientes.
- **Rol Consulta:** Perfil analítico para generación de reportes. No tiene acceso a las tablas físicas de la base de datos, garantizando que no pueda visualizar la estructura física ni alterar registros. Su acceso está confinado a Vistas Virtuales preparadas.

2. Estrategia de Vistas y Ocultamiento de Información Sensible:

- **Vista: vista_stock_simplificado**
 - *Objetivo:* Simplificar la extracción de datos.
 - *Información que protege:* Oculta la complejidad relacional de la 3FN y restringe la visibilidad de los identificadores subrogados internos



(id_producto, id_variante, etc.), exponiendo únicamente atributos entendibles por el negocio (nombre, talla, color, SKU).

- *Acceso permitido:* Rol Consulta.
- **Vista: vista_auditoria_movimientos**
 - *Objetivo:* Auditoría segura de transacciones.
 - *Información que protege (Enmascaramiento):* Evita la exposición de datos personales y fiscales sensibles. Aplica funciones de manipulación de cadenas (LEFT, SUBSTRING) para enmascarar parcialmente el Documento Nacional de Identidad (DNI) de los empleados y el CUIT financiero de los proveedores, transformando los dígitos en asteriscos y protegiendo el ecosistema frente a posibles filtraciones de información privada.
 - *Acceso permitido:* Rol Consulta.

ETAPA V

Gestión de Transacciones

Para garantizar las propiedades Atomicidad, Consistencia, Aislamiento y Durabilidad en el sistema de gestión de stock de indumentaria deportiva, se implementaron bloques transaccionales explícitos. Estos bloques envuelven la lógica de negocio (procedimientos almacenados) y aseguran que la base de datos se mantenga en un estado válido ante cualquier escenario, ya sea de éxito o de fallo crítico.

A continuación, se documentan los cuatro escenarios transaccionales desarrollados para cubrir los mecanismos exigidos:

1. Venta Exitosa en Caja (Uso de BEGIN y COMMIT)

- a. **Objetivo:** Demostrar el flujo óptimo de una transacción exitosa.
- b. **Mecanismo:** Se inicia la transacción con la instrucción BEGIN para aislar temporalmente los cambios en memoria. Se invoca el procedimiento almacenado de venta al público en la sucursal.
- c. **Resolución:** Al ejecutarse sin violar ninguna restricción (existiendo stock suficiente), el motor aprueba la operación. Finalmente, se ejecuta el comando COMMIT para escribir de forma definitiva e irreversible los cambios en el disco, descontando el stock del inventario y registrando el movimiento de "Salida".

```
-- 1. Inicia la transaccion
BEGIN;

-- 2. Consultamos el stock inicial de la variante 1 en la Sucursal 2, la del NOA
SELECT 'ANTES DE LA VENTA' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_sucursal = 2 AND id_variante = 1;

-- 3. Realizamos una venta en caja de 4 unidades de la variante 1
CALL sp_registrar_venta_caja(
    2,                -- Sucursal 2: NOA
    2,                -- Empleado 2: (Laura Diaz - CON ROL Operativo)
    'Transaccion Exitosa: Venta al publico en Catamarca',
    1,                -- Variante 1 (Pegasus 40)
    4                 -- Cantidad vendida: 4 unidades
);

-- 4. Verificamos que el stock se haya descontado correctamente dentro de la transaccion
SELECT 'DESPUES DE LA VENTA' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_sucursal = 2 AND id_variante = 1;

-- 5. Se guardan los cambios
COMMIT;
```

2. Venta Abortada por Quiebre de Stock (Uso de ROLLBACK)

- Objetivo:** Demostrar el tratamiento correcto del sistema ante un fallo lógico del negocio en este caso el intento de venta sin unidades disponibles.
- Mecanismo:** Se inicia el bloque con BEGIN y se intenta procesar una salida por una cantidad de prendas que supera el stock físico de la sucursal.
- Resolución:** El Trigger de consistencia logística detecta el faltante y lanza una excepción SQL que interrumpe el proceso. Para evitar datos corruptos o actualizaciones a medias, la transacción queda abortada. Se ejecuta explícitamente el comando ROLLBACK para deshacer cualquier acción parcial y devolver la base de datos a su estado original íntegro, protegiendo el inventario comercial.


```
-- 1. Inicia la transaccion
BEGIN;

-- 2. Consultamos el stock actual de la variante 1 en la Sucursal NOA
SELECT 'ESTADO INICIAL' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_sucursal = 2 AND id_variante = 1;

-- 3. Intentamos realizar una venta masiva (500 unidades)
-- Esto va a fallar a proposito ya que no hay suficiente stock
CALL sp_registrar_venta_caja(
    2,                -- Sucursal 2: NOA
    2,                -- Empleado 2: Laura Díaz (Operativo)
    'Intento de venta fraudulenta masiva',
    1,                -- Variante 1 (Pegasus 40)
    500              -- Cantidad: 500 unidades
);

-- 4. Ejecutamos ROLLBACK para limpiar el error y devolver la BD a su estado sin errores.
ROLLBACK;

-- 5. Verificamos que la base de datos bloquee la venta y el stock quedo intacto
SELECT 'ESTADO FINAL evitado por fallo en la transaccion' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_sucursal = 2 AND id_variante = 1;
```

3. Ingreso Masivo de Proveedor (Uso de SAVEPOINT)

- a. **Objetivo:** Evitar la pérdida de datos válidos durante la carga operativa por lotes proveniente de los proveedores.
- b. **Mecanismo:**
 - i. Se inicia la transacción (BEGIN).
 - ii. Se registra exitosamente el primer lote de zapatillas y se establece un punto de control mediante el comando SAVEPOINT lote_1.
 - iii. Al intentar ingresar el segundo lote, se simula un error de ingreso en la cantidad negativa, lo que viola la restricción CHECK de cantidades positivas, provocando un error en el motor.
- c. **Resolución:** En lugar de cancelar todo el ingreso y obligar al operario a reiniciar su trabajo, se ejecuta el comando ROLLBACK TO SAVEPOINT lote_1. Esto descarta únicamente el lote erróneo, pero rescata el trabajo previo. Finalmente, se aplica un COMMIT para guardar el lote correcto.


```
-- 1. Inicia la transaccion
BEGIN;

-- 2. Consultamos stock inicial (Ej: Variante 2 en Sucursal 1 - CABA)
SELECT 'ANTES DEL INGRESO' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_sucursal = 1 AND id_variante = 2;

-- 3. Cargamos el Lote 1 (100 unidades)
CALL sp_registrar_entrada_mercaderia(
    1, -- Sucursal 1: Central CABA
    1, -- Empleado 1: Carlos Gomez (Admin)
    1, -- Proveedor 1: Nike Argentina S.A.
    'Lote 1: Ingreso correcto de zapatillas',
    2, -- Variante 2 (Pegasus 40 talla 40 Blanco)
    100, -- Cantidad: 100 unidades
    90000.00 -- Precio de costo
);

-- 4. punto de guardado por si algo sale mal durante la transaccion
SAVEPOINT lote_1;
```

```
-- 5. Intentamos cargar el Lote 2, pero el empleado tipea mal la cantidad (-20)
-- Esto lanza un error porque viola la regla CHECK de que la cantidad debe ser mayor a cero (cantidad > 0)
CALL sp_registrar_entrada_mercaderia(
    1, 1, 1, 'Lote 2: Error en la cantidad ', 2, -20, 90000.00
);

-- 6. El Lote 2 falla En lugar de hacer un ROLLBACK total y perder el Lote 1,
-- volvemos hasta el punto de guardado.
ROLLBACK TO SAVEPOINT lote_1;

-- 7. Confirmamos definitivamente la transaccion en donde se guarda el Lote 1, se descarta el 2
COMMIT;

-- 8. Verificamos que las 100 unidades del Lote 1 si se sumaron al inventario
SELECT 'ESTADO FINAL (LOTE 1 GUARDADO)' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_sucursal = 1 AND id_variante = 2;
```

4. Traslado Logístico Inter-Sucursales (Atomicidad Compleja)

- a. **Objetivo:** Garantizar que el movimiento de indumentaria entre dos ubicaciones (Central CABA a Sucursal NOA) logrando que sea atómico.
- b. **Mecanismo:** Un traslado implica dos actualizaciones críticas simultáneas: restar stock en la sucursal de origen y sumar stock en la sucursal de destino.
- c. **Resolución:** Todo el proceso se encapsula entre un BEGIN y un COMMIT. De este modo, si ocurre un corte de red o una falla de hardware en el milisegundo exacto en que se resta la mercadería del origen, el sistema no la deja "fantasma", sino que revierte toda la operación, garantizando la consistencia del inventario en todo el ecosistema de sucursales.

```
-- 1. Inicia la transacción
BEGIN;

-- 2. Consultamos el stock en ambas sucursales antes del traslado
-- Usaremos la Variante 3 (Remera Dry-Fit Academy)
SELECT 'ANTES DEL TRASLADO' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_variante = 3 AND id_sucursal IN (1, 2)
ORDER BY id_sucursal;

-- 3. Ejecutamos el traslado de 10 unidades desde CABA (variante 1) hacia el NOA ( variante 2)
CALL sp_registrar_traslado_sucursales(
    1, -- Sucursal Origen: 1 (Central CABA)
    2, -- Sucursal Destino: 2 (Sucursal NOA)
    1, -- Empleado 1: Carlos Gómez (Admin)
    'Traslado de remeras para cubrir picos de demanda en el NOA',
    3, -- Variante 3 (Remera Dry-Fit)
    10, -- Cantidad: 10 unidades
    52000.00 -- Precio de costo
);
```

```
-- 4. Verificamos como impacto la resta y la suma en ambas sucursales a la vez
SELECT 'DURANTE EL TRASLADO' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_variante = 3 AND id_sucursal IN (1, 2)
ORDER BY id_sucursal;

-- 5. Sellamos la operacion como un bloque atomico exitoso
COMMIT;

-- 6. Verificamos el estado de la transaccion
SELECT 'ESTADO FINAL (CONFIRMADO)' AS estado, id_sucursal, id_variante, cantidad_disponible
FROM Inventario
WHERE id_variante = 3 AND id_sucursal IN (1, 2)
ORDER BY id_sucursal;
```

Manejo de concurrencia

Se realizaron pruebas con dos sesiones simultáneas de pgAdmin, representando a dos usuarios que intentaban registrar una venta sobre la misma variante y sucursal. La primera transacción bloqueó la fila correspondiente del inventario mediante **SELECT ... FOR UPDATE**.

La segunda transacción quedó en espera hasta que la primera finalizó. Al liberarse el bloqueo, la segunda operación evaluó el stock actualizado. Cuando la cantidad disponible era suficiente, la operación continuó correctamente; cuando el stock restante era inferior a la cantidad solicitada, el trigger generó una excepción y la transacción fue cancelada mediante **ROLLBACK**. De esta manera, se evitaron condiciones de carrera, cantidades negativas y movimientos inconsistentes.

Prueba con stock

Usuario 1 inicia la transacción y ejecuta el procedimiento de venta sin finalizar, lo que bloquea la fila del inventario para un posible usuario dos que intente operar sobre la misma variante de producto de la misma sucursal..

```
1  --Se conecta con el usuario usr_admin contraseña AdminCata
2  BEGIN;
3
4  CALL sp_registrar_venta_caja(
5      1,                -- Sucursal
6      1,                -- Empleado
7      'Prueba de concurrencia Administrador', -- Observaciones
8      1,                -- Variante
9      7                 -- Cantidad
10 );
11
```

Data Output	Messages	Notifications
NOTICE:	AUTOMATIZACIÓN DE PRECIOS: Se detectó precio 0.00 en Venta para la Variante 1	
NOTICE:	BÚSQUEDA EN CATÁLOGO: Producto: "Pegasus 40" Precio Oficial Encontrado: 150000.00	
NOTICE:	INYECCIÓN EXITOSA: Sobreescribiendo precio_unitario con 150000.00 antes de guardar en disco.	
NOTICE:	CONTROL DE CONCURRENCIA: Variante 1, Sucursal 1, Stock 10, Solicitado 7	
NOTICE:	STOCK BLOQUEADO Y VALIDADO CORRECTAMENTE PARA LA TRANSACCIÓN.	
NOTICE:	TRIGGER ACTIVADO: Evaluando Variante 1 en Movimiento 1006	
NOTICE:	DATOS CAPTURADOS: Tipo: Salida, Suc. Origen: 1, Suc. Destino: <NULL>	
NOTICE:	Procesando rama de SALIDA. Restando 7 unidades...	
NOTICE:	✅ VENTA REGISTRADA EXITOSAMENTE: Movimiento N° 1006 en Sucursal 1	
CALL		

Usuario 2 inicia una transacción con los mismos registros, la cual estará bloqueada mientras el Usuario 1 no finalice la transacción anterior

```
1  --Se conecta como usr_caja contraseña OperativoStock
2  BEGIN;
3
4  CALL sp_registrar_venta_caja(1, 2, 'Prueba de concurrencia U2', 1, 7);
5
```

Data Output	Messages	Notifications
NOTICE:	AUTOMATIZACIÓN DE PRECIOS: Se detectó precio 0.00 en Venta para la Variante 1	
NOTICE:	BÚSQUEDA EN CATÁLOGO: Producto: "Pegasus 40" Precio Oficial Encontrado: 150000.00	
NOTICE:	INYECCIÓN EXITOSA: Sobreescribiendo precio_unitario con 150000.00 antes de guardar en disco.	

Donde una vez realizado el respectivo **COMMIT** en la transacción iniciada por el Usuario 1 donde vamos a tener 2 casos.

1- Stock insuficiente donde la operación es rechazada por el trigger y la transacción queda abortada hasta ejecutar un **ROLLBACK**.

```

Query  Query History
1  --Se conecta como usr caja contraseña OperativoStock
2  BEGIN;
3
4  CALL sp_registrar_venta_caja(1, 2, 'Prueba de concurrencia U2', 1, 7);
5

Data Output  Messages  Notifications
NOTICE: AUTOMATIZACIÓN DE PRECIOS: Se detectó precio 0.00 en Venta para la Variante 1
NOTICE: BÚSQUEDA EN CATÁLOGO: Producto: "Pegasus 40" | Precio Oficial Encontrado: 150000.00
NOTICE: INYECCIÓN EXITOSA: Sobreescribiendo precio_unitario con 150000.00 antes de guardar en disco.
NOTICE: CONTROL DE CONCURRENCIA: Variante 1, Sucursal 1, Stock 3, Solicitado 7
ERROR:  ✖ ERROR OPERATIVO EN VENTA DE CAJA: STOCK INSUFICIENTE. Variante: 1, Sucursal: 1, Disponible: 3, Solicitado: 7

```

2- Stock suficiente donde finaliza correctamente la transacción pudiendo descontar el stock.

```

1  --Se conecta como usr caja contraseña OperativoStock
2  BEGIN;
3
4  CALL sp_registrar_venta_caja(1, 2, 'Prueba de concurrencia U2', 1, 7);
5

Data Output  Messages  Notifications
NOTICE: AUTOMATIZACIÓN DE PRECIOS: Se detectó precio 0.00 en Venta para la Variante 1
NOTICE: BÚSQUEDA EN CATÁLOGO: Producto: "Pegasus 40" | Precio Oficial Encontrado: 150000.00
NOTICE: INYECCIÓN EXITOSA: Sobreescribiendo precio_unitario con 150000.00 antes de guardar en disco.
NOTICE: CONTROL DE CONCURRENCIA: Variante 1, Sucursal 1, Stock 7, Solicitado 7
NOTICE: STOCK BLOQUEADO Y VALIDADO CORRECTAMENTE PARA LA TRANSACCIÓN.
NOTICE: TRIGGER ACTIVADO: Evaluando Variante 1 en Movimiento 1009
NOTICE: DATOS CAPTURADOS: Tipo: Salida, Suc. Origen: 1, Suc. Destino: <NULL>
NOTICE: Procesando rama de SALIDA. Restando 7 unidades...
NOTICE:  ✔ VENTA REGISTRADA EXITOSAMENTE: Movimiento N° 1009 en Sucursal 1

```